Latest updates: https://dl.acm.org/doi/10.1145/3769831

PDF Download
3769831.pdf
07 January 2026
Total Citations: 0
Total Downloads: 138

Published: 05 December 2025

Citation in BibTeX format

RESEARCH-ARTICLE

AixelAsk: A Stepwise-Guided Retrieval and Reasoning Framework for Large Table QA

CHI ZHANG, Beijing Institute of Technology, Beijing, China

MEIHUI ZHANG, Beijing Institute of Technology, Beijing, China

YUXIN YANG, Beijing Institute of Technology, Beijing, China

TAO CHEN, Beijing Institute of Technology, Beijing, China

ZHAOJING LUO, Beijing Institute of Technology, Beijing, China

Open Access Support provided by:

Beijing Institute of Technology

Proceedings of the ACM on Management of Data, Volume 3, Issue 6 (December 2025)
https://doi.org/10.1145/3769831
EISSN: 2836-6573

AixelAsk: A Stepwise-Guided Retrieval and Reasoning Framework for Large Table QA

CHI ZHANG, Beijing Institute of Technology, P. R. China
MEIHUI ZHANG*, Beijing Institute of Technology, P. R. China
YUXIN YANG, Beijing Institute of Technology, P. R. China
TAO CHEN, Beijing Institute of Technology, P. R. China
ZHAOJING LUO, Beijing Institute of Technology, P. R. China

In the big data era, Table Question Answering (Table QA) has emerged as a crucial tool for extracting insights from structured data, especially in large table scenarios. There are two main categories of methods for Table QA: Executable Code-driven methods and Language Model based (LM-based) methods. Code-driven methods, e.g. Text-to-SQL based solutions, often struggle with incomplete or mismatching schema information. LM-based methods, include Pre-trained Language Models (PLMs) and Large Language Models (LLMs), also face challenges as PLMs have limited generalization, while LLMs suffer from performance degradation and increased token cost when applied to large tables.

To address these challenges, we propose AixelAsk, a novel LLM-based framework designed for Large Table QA. Specifically, AixelAsk incorporates a three-module architecture consisting of Decomposition module, Retrieval module and Reasoning module. The Decomposition module constructs a directed acyclic graph (DAG)-based solution plan by decomposing the question into execution nodes with explicit dependencies, making a clear reasoning path to guide the LLM through a logical process. Inspired by the Retrieval-Augmented Generation, the Retrieval Module extracts key rows and columns from the large table, reducing input token size and focusing on critical information. The Reasoning Module performs step-by-step inferences over the retrieved sub-tables, guided by each execution node in the solution plan, to generate final answer. By tackling the challenges of LLM performance degradation with large inputs and complex questions, AixelAsk achieves superior performance in Large Table QA. Extensive experiments on various baselines across three datasets demonstrate the effectiveness and efficiency of our proposed AixelAsk framework. AixelAsk outperforms the state-of-the-art baseline by 4% - 8% in the exact match score, and at the same time reduces token usage by 86.4%, achieving both high accuracy and cost efficiency in the Large Table QA task.

CCS Concepts: • **Computing methodologies** → *Information extraction*; • **Information systems** → **Data management systems**.

Additional Key Words and Phrases: Table Question Answering, Large Language Model, Structured Data

ACM Reference Format:
Chi Zhang, Meihui Zhang, Yuxin Yang, Tao Chen, and Zhaojing Luo. 2025. AixelAsk: A Stepwise-Guided Retrieval and Reasoning Framework for Large Table QA. *Proc. ACM Manag. Data* 3, 6 (SIGMOD), Article 366 (December 2025), 25 pages. https://doi.org/10.1145/3769831

*Corresponding Author

Authors' Contact Information: Chi Zhang, Beijing Institute of Technology, P. R. China, chi_zhang@bit.edu.cn; Meihui Zhang, Beijing Institute of Technology, P. R. China, meihui_zhang@bit.edu.cn; Yuxin Yang, Beijing Institute of Technology, P. R. China, yuxin_yang@bit.edu.cn; Tao Chen, Beijing Institute of Technology, P. R. China, tao_chen@bit.edu.cn; Zhaojing Luo, Beijing Institute of Technology, P. R. China, zjluo@bit.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2836-6573/2025/12-ART366
https://doi.org/10.1145/3769831

1 Introduction

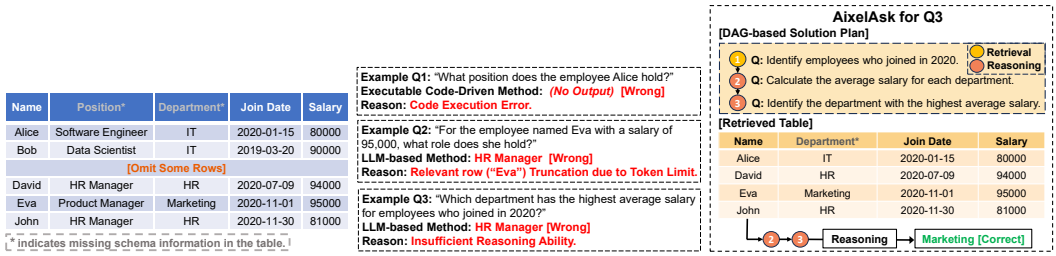
With the advent of big data, tables have become a fundamental tool for managing structured data, widely used across domains such as business analytics, scientific research, and government statistics [5, 21, 23, 30, 33, 49]. As the cornerstone of structured data analysis [34, 53], tables provide an essential foundation for organizing, exploring, and deriving insights from structured information [31, 32]. In order to facilitate the extraction of meaningful insights from tabular data, the task of table question answering (Table QA) has been introduced. Table QA [7, 25, 39, 58, 61, 62] focuses on answering natural language questions based on tabular data, facilitating interaction with structured information.

In real-world applications, tables are becoming more diverse in structure and content, and their size is rapidly increasing. Large tables often contain numerous rows and columns, making it difficult to identify the key regions required for reasoning [1, 3]. For example, in a table with hundreds of rows, only a handful of rows may be relevant for answering the question. Additionally, as Table QA systems are applied to large tables, the questions posed often become more complex, requiring multi-step reasoning and the synthesis of information from different parts of the table to draw accurate conclusions [10]. The increasing size of tables and complexity of questions call for more effective and efficient methods to handle the Large Table QA task.

However, research on Large Table QA is still limited. Some methods adopt the Executable Code-driven approach [6, 14, 22, 44], which translates natural language questions into executable programs such as SQL or Python, and executes to generate answers. While effective in certain scenarios, these approaches heavily depend on the table's schema. As an example, Figure 1a shows a large table with incomplete schema information. The Executable Code-driven method fails on Q1 (in Figure 1b) due to the absence of the *"Position"* column information. Besides schema incompleteness, ambiguities between the question and the table schema [16, 50] can also lead to incorrect code generation, resulting in failures to access the table.

Some studies turn to Language Model (LM)-based Table QA methods, which directly serialize the table and input it into LMs. Such methods enable fine-grained comprehension of table content, thereby reducing the dependency on schema knowledge. LM-based methods can be categorized into Pre-trained Language Model (PLM)-based approaches [15, 28, 55] and Large Language Model (LLM)-based approaches [48, 54, 60]. PLM-based methods require task-specific fine-tuning and face challenges in handling large tables due to limited context modeling capabilities. LLM-based methods do not require fine-tuning but struggle with large tables and complex queries, leading to performance degradation [42]. Chain-of-Table [52] assumes full-table access but struggles to scale when token limits are exceeded, resulting in table truncation and reduced accuracy. As shown in Figure 1b, LLM-based methods fail to answer Q2 due to token truncation, which excludes critical row mentioning *"Eva"*. Dater [54] decomposes problems into simple sub-questions but still fails to handle multi-step reasoning tasks, where its reasoning ability falls short. In Figure 1b, LLM-based methods fail on Q3, which requires multi-step reasoning beyond their capabilities. Additionally, ReAcTable [59] requires multiple LLM invocations for voting, leading to high token consumption and increased computation time. Although LLM-based methods perform the best among existing works, they still face significant challenges in accuracy and efficiency when dealing with large tables and complex QA tasks.

Challenge 1: One of the major challenges comes from the large tables. Large tables often exceed the token limits of LLMs and introduce substantial irrelevant content, which leads to performance degradation. Current LLM-based methods obtain reduced performances as the token length increases [27, 56]. An ideal framework of Large Table QA should be capable of dynamically selecting and focusing on the most relevant table regions, rather than treating the entire table



(a) A Large Table of Employee Information (b) Failure Cases of Existing Methods (c) Our Proposed AixelAsk Solution

Fig. 1. Comparison of Various Table QA Methods

uniformly. However, this is challenging because most tokens in large tables are irrelevant to the question, making it difficult for the model to identify truly informative data. Furthermore, relevant information is often dispersed rather than contiguous, demanding structured, reasoning-aware selection strategies. This highlights the importance of a fine-grained retrieval strategy capable of selectively identifying the most relevant regions of the table with respect to the question.

Challenge 2: Another key challenge is handling complex questions in Large Table QA, which often involves multi-step reasoning. Most methods either reason in a single step [52, 59], overloading the model's capacity, or decompose queries without properly considering execution dependencies [54]. An ideal framework should decompose questions into logically structured sub-questions that enable clear step-by-step reasoning. However, achieving this decomposition is challenging. It demands accurately identifying sub-questions that clearly isolate individual reasoning steps while preserving logical connections. This calls for a decomposition mechanism tailored to Large Table QA that enables effective multi-step reasoning.

Challenge 3: There lacks a cost-efficient strategy when using LLMs in Table QA. Existing works [54, 59] rely on multiple LLM calls, resulting in high token usage and increased inference overhead. Ideally, an efficient framework should reduce token consumption and limit LLM invocation without compromising reasoning accuracy. However, achieving such efficiency is challenging. It requires not only filtering out irrelevant information to minimize token input, but also reducing the frequency of LLM invocations through efficient task structuring and execution planning. In multi-step reasoning, a major challenge lies in avoiding repeated inclusion of previously processed context while preserving logical coherence between sub-questions. Therefore, reducing token usage and avoiding unnecessary LLM invocations while maintaining logical consistency and accuracy remains a critical challenge in achieving efficiency for the Large Table QA task.

To address the challenges of Large Table QA, we propose a three-module framework AixelAsk[1] that integrates directed acyclic graph (DAG)-based stepwise reasoning with a row-column level retrieval strategy. Specifically, the Decomposition Module constructs a DAG-based solution plan that decomposes the question into multiple execution nodes, explicitly modeling the reasoning flow and stepwise dependency. The Retrieval Module, inspired by the RAG paradigm, applies a row-column level ensemble strategy to identify relevant table regions for retrieval nodes, effectively reducing input tokens while preserving critical information. The Reasoning Module performs lightweight inference and generates the final answer by reusing intermediate results, supporting accurate and efficient multi-step reasoning. As shown in Figure 1c, to answer Q3, DAG-based decomposition forms a workflow of filtering, calculating, and comparing. Combined with targeted

[1]The source code, prompts and datasets of AixelAsk are available at: https://github.com/ChiZhang-bit/AixelAsk.

sub-table retrieval, AixelAsk performs step reasoning and derives the correct answer *"Marketin"*. AixelAsk is a component of Aixel [57], a unified, adaptive and extensible system for AI-powered data analysis. It has been integrated into Aixel as its dedicated task layer for Large Table QA, supporting efficient and accurate execution of large-table question answering within the system.

In summary, our main contributions are as follows:

- We propose the AixelAsk framework that decomposes the question, integrates retrieval and reasoning, to handle Large Table QA more accurately and efficiently.
- We design a DAG-based decomposition method that generates a structured solution plan, supporting stepwise reasoning with explicit execution dependencies.
- We design a row-column level ensemble retrieval strategy to identify key table regions, reducing token length and cost while improving reasoning accuracy.
- Extensive experiments on three datasets demonstrate that our proposed AixelAsk improves accuracy by up to 8.24%, reduces token usage by 86.4%, while also improving time efficiency, achieving both high accuracy and cost-efficiency in Large Table QA.

In the remainder of this paper, we first review related works in Section 2. The details of our proposed AixelAsk are elaborated in Section 3. In Section 4, we present the experimental results. Finally, in Section 5, we conclude this paper.

2 Related Work

2.1 Table QA

2.1.1 Code-driven Methods. One common approach for Table QA task is code-driven methods based on table schema information. This approach converts natural language questions into executable code to retrieve answers from the table. Text-to-SQL [63] methods translate natural language questions into SQL queries, enabling direct execution in databases to obtain answers. Building upon this, TabSQLify [37] further improves precision by extracting relevant sub-tables from larger tables through text-to-SQL generation. DIN-SQL [43] advances this approach by decomposing the SQL generation process into smaller sub-problems, significantly enhancing the reasoning capabilities and accuracy of generated SQL queries. Text-to-Python method [6] uses few-shot prompting to translate NL questions into Python programs, which are executable on Pandas data frames. Also, some methods combine SQL and Python code to facilitate reasoning, such as ReAcTable [59] and Binder [12]. However, these code-driven methods are highly dependent on the table schema. When the schema is incomplete or ambiguous [16, 50], the generated code may fail to execute on the table, further degrading the performance of these methods. In contrast, our method avoids explicit code generation and instead adopts a schema-light, reasoning-driven framework to solve large table scenarios.

2.1.2 LM-based Methods. Another approach focuses on LM-based methods. TaPas [15], TAPEX [28], and TaBERT [55] all leverage pretrained models to enhance table understanding and reasoning by encoding structured table data and aligning it with natural language queries for effective table QA. However, pre-trained methods are limited by their training datasets and require fine-tuning for different types of Table QA tasks. With the advancement of LLMs, some studies have attempted to replace pre-trained models with LLMs, utilizing in-context learning to solve problems [9, 35]. Nevertheless, when the table token length increases, the model's reasoning performance degrades significantly. To address this limitation, Dater [54] introduced the concept of sub-evidence, extracting relevant sub-tables for a given question. However, this method uses LLMs as decomposers to select rows and columns, still requiring the entire table to be input into the model. Chain-of-Table [52] explicitly incorporates tabular data into the reasoning process by iteratively updating

intermediate tables, enabling dynamic step-by-step planning and improved accuracy. For large tables with thousands of tokens, Dater and Chain-of-Table are ineffective in reducing token cost and time consumption. Our work extends LM-based approaches with a scalable framework better suited for large table scenarios. By leveraging a declarative pipeline, DocETL [46] provides an LLM agent-driven framework that decomposes complex document processing tasks, optimizes the execution, and improves reasoning accuracy. Similar with the decomposition-based idea, LOTUS [41] enhances multi-step Table QA by verifying each step through a table executor, improving faithfulness. In a broader context, Palimpzest [26] applies a declarative optimization framework to AI workloads, selecting execution plans that trade off efficiency and quality. Focusing more on execution optimizations, Galois [45] introduces a hybrid query engine that integrates LLMs with logical and physical database optimizations, enhancing the accuracy and cost-efficiency of SQL execution over LLMs. Rather than following a declarative framework, our method introduces a dedicated framework for Large Table QA.

2.2 RAG on QA Tasks

Retrieval-Augmented Generation (RAG) is a framework that combines a retrieval mechanism with a generative model, enabling language models to incorporate external knowledge to enhance query answering. RAG has been extensively applied to document QA, where it retrieves contextually relevant text passages from a knowledge base to enhance the generative model's ability to produce accurate and well-informed answers [4, 17, 18, 47]. Beyond document-level retrieval, RAG techniques have been extended to various domains, including multi-modal retrieval (e.g., text and images) [13, 19], database question answering [2] and knowledge graph question answering [36]. These adaptations highlight the versatility of RAG in leveraging structured and unstructured knowledge across different modalities and data formats.

However, existing RAG methods primarily focus on retrieving information at coarser-grained levels, such as documents, passages, and images. While effective for a variety of tasks, these methods lack the ability to perform fine-grained retrieval and reasoning on structured data, particularly at the row and column levels of tables. Recognizing the similarity between these scenarios and large TableQA, we design a RAG framework to address the challenges of structured tables by adapting its approach for fine-grained retrieval and reasoning specific to rows and columns. This design ensures precise selection of relevant data, improving accuracy and cost-efficiency for the Large Table QA task.

Specifically, we design the RAG framework to operate on structured tabular data. Instead of retrieving text passages, we perform fine-grained retrieval at the row and column level of large tables. This tailored approach addresses the unique challenges of processing large tables, including large sizes and structural complexity, enabling efficient and accurate retrieval and reasoning for the Large Table QA task.

3 Methodology

To address the challenges of performance degradation in LLMs with increasing token input and the inability to handle complex questions in Large Table QA, we design the AixelAsk framework.

The overall architecture of our framework is shown in Figure 2, which comprises three components: (a) **Decomposition Module** decomposes complex questions into a directed acyclic graph, with each node representing a distinct execution step. In parallel, the module partitions the table into independent row and column chunks, laying the foundation for targeted retrieval in subsequent stages. (b) **Retrieval Module** employs a row-column level ensemble retrieval strategy to select relevant rows and columns, which are mapped back into a compact sub-table for downstream reasoning. (c) **Reasoning Module** integrates the sub-question and the states of its predecessor

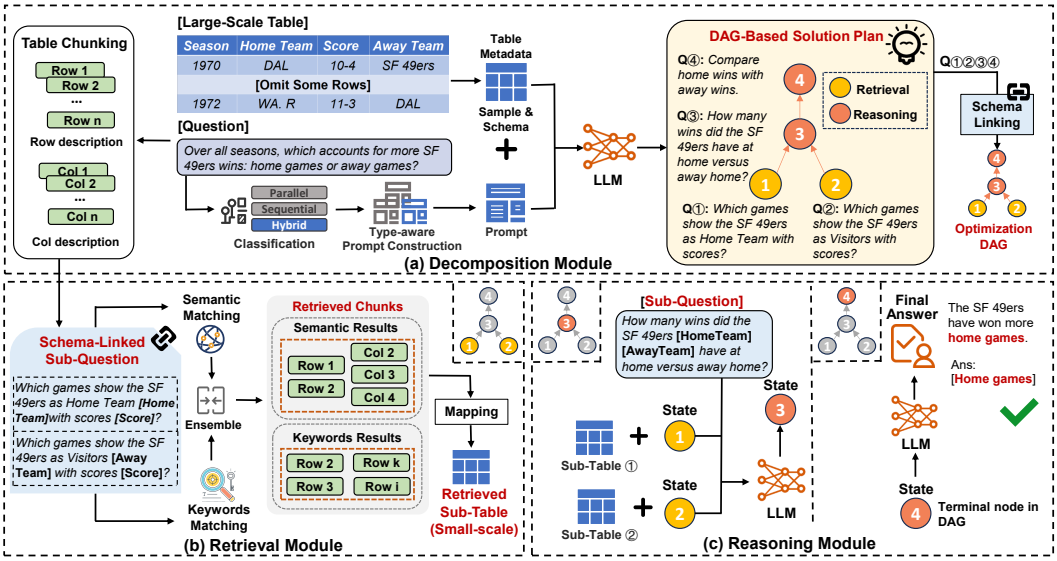


Fig. 2. The overview of AixelAsk framework, which is composed of three modules: Decomposition Module, Retrieval Module and Reasoning Module.

nodes to perform inference. After all reasoning nodes are executed, the final answer is generated based on the terminal nodes in the DAG.

Before providing a detailed explanation of our modules, we first define the key concepts and the problem addressed in this paper.

1) *Large Table:* A *large table* is of structured relational data format, and typically contains massive amounts of data, exceeding the processing capacities of traditional Table QA methods. Formally, a large table T can be represented as $T = (S, D)$ where:

- S denotes the schema (headers) of the table, formally defined as $S = \text{Schema}(T)$.
- D denotes the set of data elements in the table, represented as $D = \{d_{ij} | i \in [1, M], j \in [1, N]\}$ where M and N indicate the total number of rows and columns. Each data element d_{ij} represents the content at the intersection of the i-th row and j-th column.

2) *Large Table QA:* Given a single large table T and a natural language question Q related to the data within T, the goal of our Large Table QA task is to generate the correct answer A. The question Q ranges from simple lookups to complex queries that implicitly involve operations such as group-by, order-by, aggregation, or conditional filtering.

The following sections present a detailed introduction to the three modules of our framework.

3.1 Decomposition Module

The Decomposition Module is designed to tackle the challenges of handling complex questions in the Large Table QA task. It consists of two primary components: the generation of the solution plan and the decomposition of the table. The solution plan decomposes a complex question Q into a series of sub-level questions and corresponding actions, represented as a Directed Acyclic Graph (DAG) to capture the dependencies between sub-questions. Meanwhile, the table decomposition component partitions the table into row and column segments, which are transformed into independent natural

language chunks. The decomposition enhances retrieval efficiency and enables more effective reasoning in subsequent stages.

3.1.1 ***DAG-based Solution Plan Construction.*** To handle the complexity of multi-step reasoning in Large Table QA, we adopt the question decomposition. Instead of relying on simple linear decomposition, DAG-based decomposition aims to separate evidence retrieval from reasoning in a more fine-grained manner by isolating reasoning steps and explicitly modeling their logical and execution dependencies. To this end, we design DAG-based solution plan construction, which provides a structured and coherent organization of the decomposition. Unlike CoT prompting [51], which produces intermediate reasoning steps in an implicit and unstructured manner, DAG can explicitly encode logical and execution dependencies between sub-questions in a structured form. Consequently, DAG-based solution plan design supports modular reasoning with clearer logic flow, especially for complex queries requiring non-linear reasoning paths.

When generating the solution plan using LLMs, it is essential to provide the model with relevant information about the table. To reduce the number of tokens required for understanding large tables, we define the table's metadata as:

$$M(T) = \{S, T_{sample}\} \tag{1}$$

Here, S denotes the table schema and T_{sample} is a subset of rows randomly sampled three times from the full table T. The metadata $M(T)$ provides a compact and informative representation of the table.

To guide the generation of the solution plan, we introduce a classification that determines the reasoning logic of each question. Specifically, we fine-tune a lightweight classifier based on RoBERTa[29] to categorize questions into three types:

- **Parallel**: Sub-questions can be solved independently, forming parallel branches in the DAG.
- **Sequential**: Sub-questions follow a linear chain, each depending on the previous one.
- **Hybrid**: Combines parallel branches and merge points with sequential dependencies.

For each question type, we design a specific prompt template and construct several representative in-context examples tailored to its corresponding reasoning pattern. In Figure 2, the example question is classified as *hybrid* because it involves parallel analysis of *Home and Away Team* cases followed by a comparison.

Next, the original question Q combined with the metadata $M(T)$ is incorporated into the selected prompt template and fed into an LLM using few-shot prompting, guiding the model to explicitly separate the retrieval and reasoning processes. The LLM identifies sub-questions and their dependencies, constructing nodes and edges to form a complete DAG representing the workflow. In Figure 2, the question is decomposed into a DAG with two retrieval nodes for the *"Home Team"* and *"Away Team"*, followed by two reasoning nodes that infer the answer based on the retrieved subtables. We formalize this procedure as the decomposition operator D_{DAG}, generating the solution plan represented by a directed acyclic graph:

$$G = D_{\text{DAG}}(Q, M(T)) = (V, E) \tag{2}$$

In this representation, V denotes the set of all sub-tasks (sub-level questions), formally defined as:

$$V = \{v_1, v_2, \ldots, v_{|V|}\} \tag{3}$$

Here, $|V|$ denotes the number of execution nodes. Each node $v_i \in V$ is composed of a natural language sub-question and its corresponding action:

$$v_i = (Q_i, \alpha_i), \quad \alpha_i \in \{\text{Retrieval}, \text{Reasoning}\} \tag{4}$$

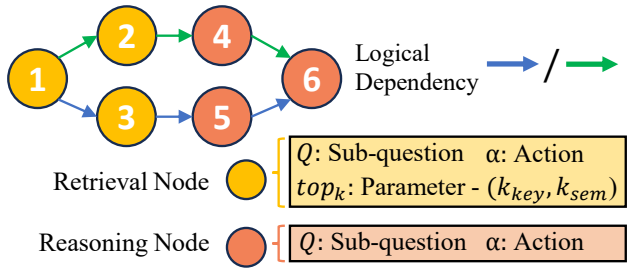


Fig. 3. An Example of DAG Structure in Solution Plan

where Q_i represents the decomposed sub-question at node v_i, and α_i specifies the action type: either Retrieval or Reasoning. Retrieval nodes are parameterized by a dynamically generated top-k (k_{sem} and k_{key}) value, which controls the number of rows or columns to retrieve. Reasoning nodes require no additional parameters and operate on the outputs of preceding nodes. Moreover, the set of directed edges E represents computational dependencies among nodes, formally defined as:

$$E \subseteq V \times V, \quad (u, v) \in E \tag{5}$$

where each edge (u, v) indicates that the computation at node v explicitly relies on the output generated from node u.

To ensure the correctness of the generated execution logic, we perform a structural validation on the generation. Specifically, we verify whether the graph $G = (V, E)$ forms a valid DAG. A DAG is considered invalid if it violates acyclicity constraints, lacks required parameters, or contains malformed node or edge definitions. Upon detecting an invalid DAG, we automatically regenerate it, retrying up to a fixed number of attempts.

Optimization via Schema Linking. During DAG Solution Plan generation, the LLM decomposes the original question into multiple sub-questions. However, sub-questions affected by ambiguity, incompleteness, or implicit references often lead to reduced retrieval accuracy in the subsequent Retrieval Module. To address these limitations, we introduce a schema-aware strategy based on Schema Linking, which explicitly aligns each sub-question with the table schema to produce refined sub-questions. For each node $v_i = (Q_i, \alpha_i)$ in the G, we define a schema operator $\mathcal{L}$ that takes the sub-question Q_i and the table schema S as input, and outputs a refined sub-question $\tilde{Q}_i$:

$$\tilde{Q}_i = \mathcal{L}(Q_i, S) \tag{6}$$

The schema linking $\mathcal{L}$ involves three steps. First, we extract potential entities from the sub-question using the LLM with zero-shot prompting.

$$\mathcal{E}(Q_i) = \varepsilon_1, \varepsilon_2, \ldots, \varepsilon_\ell \tag{7}$$

For each extracted entity, we prompt the LLM to decide whether it should be linked to any schema element, based on their semantic compatibility.

$$\text{Link}(\varepsilon, S) \subseteq S \tag{8}$$

Finally, we augment the sub-question by inserting the linked schema element at the position of identified entity, forming a schema-aligned question $\tilde{Q}_i$.

$$\tilde{Q}_i = Q_i \oplus \left(\bigcup_{\varepsilon \in \mathcal{E}(Q_i)} \text{Link}(\varepsilon, S) \right) \tag{9}$$

where the operator $\oplus$ denotes a fusion operation that appends relevant schema context to the sub-question.

In Figure 2, the sub-question in execution node 2 contains the entity *"Visitor"*, which is ambiguous and may hinder effective retrieval. Through schema linking, *"Visitor"* is correctly mapped to the schema *"Away Team"* and integrated into the sub-question. Schema Linking resolves ambiguity and enhances semantic clarity, providing a better foundation for subsequent retrieval and reasoning.

Execution Strategy Guided by DAG. The DAG not only reflects the reasoning structure but also serves as a blueprint for determining execution order. Nodes are executed sequentially according to their dependencies in the DAG, while those without dependencies can be executed in parallel. As shown in Figure 3, the DAG defines the workflow by capturing dependencies between execution stages. Node 2, 3 should be executed after node 1 completed. The green and blue paths are independent, so node 2,4 and node 3,5 can be executed in parallel after node 1 completed. The hybrid execution plan improves overall efficiency while preserving the logic of the multi-step reasoning process.

Execution Optimizations Opportunities. While the execution strategy supports parallelism where applicable, there remains room for further optimization. Specifically, reducing query complexity can be used as a proxy for improving execution efficiency. The number of nodes in the DAG (i.e., DAG size) can be an indicator for complexity, and thus reducing DAG size via node merges can result in better efficiency. However, node merging essentially combines multiple retrieval or reasoning steps, requiring a single LLM invocation to handle multiple tasks, which may affect answer quality. The optimization goal is to minimize DAG size under certain answer quality constraints to achieve a trade-off between execution efficiency and effectiveness. There are two possible ways to merge the nodes. One way is to merge multiple compatible execution nodes that are independent and share a common preceding node. For example, in Figure 3, nodes 2 and 3 can be jointly executed in a LLM call. Additionally, sequential nodes can also be merged. In Figure 3, node pairs such as (2, 4) or (3, 5) could be combined in this manner. These two types of optimizations must be applied under the premise that they do not significantly impact answer quality. It is necessary to define constraints that guide when and how many nodes can be merged without compromising the answer quality. Possible constraints include limiting the number of merging nodes based on certain conditions, e.g., the total input token length and the number of reasoning tasks, to avoid overloading LLM with too many objectives. Additionally, merging nodes should be semantically related to prevent mixing unrelated tasks within one prompt. These optimization strategies offer promising directions for improving AixelAsk's execution efficiency in our future work.

The DAG-based solution plan provides a clear and structured representation of dependencies among sub-questions, enabling systematic and accurate step-by-step reasoning for complex Table QA tasks.

3.1.2 ***Row-Column Level Table Chunking.*** The core objective of table chunking is to decompose a table into multiple independent data chunks by converting each row and column into a natural language description. Formally, given a table $T = (S, D)$, we define the original row and column sets as $R = \{r_i | i \in [1, M]\}$ and $C = \{c_j | j \in [1, N]\}$. Specifically, each row r_i and column c_j can be independently transformed into natural language, generating distinct descriptive chunks. Converting rows and columns into descriptions chunks enables language models to efficiently retrieve relevant information.

Row Description. Each row r_i in the table contains multiple values. To convert rows into natural language, we combine the S with the data values of r_i, generating description $Des(S, r_i)$ using row template generated by the LLM.

To reduce the computational cost of processing large tables, we generate a table-specific row template for each table by providing its metadata $M(T)$ to the LLM in a single one-shot prompt. This template is a parameterized sentence constructed from the table's schema. Each row description is generated by substituting row values into the parameterized table-specific row template. For example, in Figure 2, using the schema *Season*, *Home Team*, *Score* and *Away Team*, with the values *1970*, *Dallas Cowboys*, *10-4* and *SF 49ers*, the description would be:

> *"In the 1970 season, the Dallas Cowboys as Home Team won SF 49ers as Away Team with a score of 10-4."*

Column Description. Column descriptions are generated to capture the semantic meanings of each column. Formally, for column c_j, we sample a small subset of table data and feed it to the language model. The resulting column description can be represented as $Des(c_j)$. These descriptions significantly enhance semantic retrieval by clearly expressing column semantics.

After obtaining the natural language descriptions for both rows and columns, we treat each description as an independent data chunk. The final collection of data chunks is formally represented as:

$$C(T) = \{Des(S, r_i) \mid i \in [1, M]\} \cup \{Des(c_j) \mid j \in [1, N]\} \tag{10}$$

$C(T)$ collectively denotes all descriptive chunks of the original table, serving as a structured natural language representation that facilitates efficient retrieval and reasoning for downstream tasks.

3.2 Retrieval Module

The Retrieval Module identifies relevant regions of the table required to answer a specific sub-question. The module is activated for each execution node $v_i = (\tilde{Q}_i, \alpha_i)$ in the DAG-based solution plan when the associated action α_i is *Retrieval*. To retrieve relevant sub-table for each execution node, we design a **row-column level ensemble retrieval strategy** that combines semantic similarity and keyword-based matching to select the most relevant data chunks. As a prerequisite, the original table T is partitioned into independent data chunks, denoted as $C(T)$, which include natural language descriptions of both rows and columns. Retrieval is then performed over $C(T)$ to identify a set of relevant rows and columns, forming a question-specific sub-table through row-column level retrieval.

3.2.1 ***Semantic Similarity Matching.*** In the semantic similarity matching phase, we utilize large language embedding models to project both sub-questions and data chunks into a shared high-dimensional semantic space. Formally, for each chunk τ in the chunk collection $C(T)$, we compute its embedding as:

$$e_\tau = \phi(\tau), \quad \forall \tau \in C(T) \tag{11}$$

where $\phi(\cdot)$ denotes the embedding function provided by a embedding model, such as OpenAI's `text-embedding-3-large`. Similarly, for the schema-linked sub-question $\tilde{Q}_i$ associated with execution node v_i, we compute its embedding using the same embedding function:

$$e_i = \phi(\tilde{Q}_i) \tag{12}$$

We then perform row-column level semantic matching by independently computing the cosine similarity between the sub-question embedding and each chunk embedding. Formally, for chunk $\tau \in C(T)$, the similarity score is:

$$\text{sim}(\tilde{Q}_i, \tau) = \cos(e_i, e_\tau) = \frac{e_i \cdot e_\tau}{|e_i| \cdot |e_\tau|}, \quad \forall \tau \in C(T) \tag{13}$$

To obtain a soft selection over chunks, we introduce a temperature parameter t_0 and compute a probability distribution over all chunks based on their similarity scores:

$$P_{\text{sem}}(\tau \mid \tilde{Q}_i) = \frac{\exp\left(\text{sim}(\tilde{Q}i, \tau)/t_0\right)}{\sum_{\tau' \in C(T)} \exp\left(\text{sim}(\tilde{Q}_i, \tau')/t_0\right)} \tag{14}$$

$P_{\text{sem}}(\tau \mid \tilde{Q}_i)$ represents the relevance probability of chunk τ with respect to the sub-question $\tilde{Q}_i$. We then select the top-$k$sem chunks as the semantic retrieval result for Q_i:

$$R_i^{\text{sem}} = \text{TopK}(P_{\text{sem}}(\cdot \mid \tilde{Q}_i),\ k_{\text{sem}}) \tag{15}$$

3.2.2 ***Keywords Matching.*** Keywords matching is designed to capture table chunks that may not achieve high scores in the semantic embedding space but have direct lexical associations with the sub-level question. We define a keyword extraction function $\mathcal{K}(\cdot)$ that extracts a set of keywords from sub-questions and chunks, typically including nouns, proper nouns and other semantically salient terms. For the sub-question $\tilde{Q}_i$ and a chunk τ, we compute the keyword matching score based on the overlap between their extracted keyword sets:

$$\sigma(\tilde{Q}_i, \tau) = \left|\mathcal{K}(\tilde{Q}_i) \cap \mathcal{K}(\tau)\right| \tag{16}$$

The keyword matching score reflects the degree of lexical overlap between the sub-question and the chunk. Based on the keyword matching score, we select the top-k_{key} chunks with the highest lexical overlap as the keyword-based retrieval result:

$$R_i^{\text{key}} = \text{TopK}(\{\tau \mid \sigma(\tilde{Q}_i, \tau)\},\ k_{\text{key}}) \tag{17}$$

3.2.3 ***Ensemble Retrieval.*** After obtaining the candidate chunks from both semantic matching and keyword matching, we perform ensemble retrieval by merging the two sets. The final chunk set R_i for sub-question Q_i is defined as:

$$R_i = R_i^{\text{sem}} \cup R_i^{\text{key}} \tag{18}$$

With the unified set of relevant chunks R_i, we map the selected row and column descriptions back to their corresponding positions in the original table T. This mapping is performed via a function $\bar{M}()$ that reconstructs the retrieved table region Γ_i associated with execution node v_i:

$$\Gamma_i = \bar{M}(R_i, T) \tag{19}$$

The region Γ_i serves as the targeted sub-table for downstream reasoning and execution.

The Retrieval Module filters the large table into a smaller, focused sub-table, reducing the input size for downstream reasoning. Row-column level retrieval enables fine-grained control over table selection, avoiding irrelevant information and enhancing interpretability. By combining semantic and lexical signals, the ensemble retrieval strategy ensures the retrieved content is both relevant and precise, thereby improving the accuracy of subsequent reasoning.

3.3 Reasoning Module

Following the construction of the solution plan and the identification of relevant regions within the table, the Reasoning Module executes the reasoning steps required to derive intermediate or final answers. The module is activated for each execution node $v_i = (\tilde{Q}_i, \alpha_i)$ in the DAG-based solution plan when the associated action α_i is *Reasoning*.

To perform structured multi-step reasoning within the DAG, the Reasoning Module computes the state of each reasoning node based on its sub-question, the results from its predecessor nodes,

and the associated retrieved table regions. Formally, for each reasoning node v_i, its execution state S_i is defined as:

$$S_i = \mathcal{G}\Big(\tilde{Q}_i, \bigoplus_{u \in \text{pred}(v_i)} S_u, \bigcup_{u \in \text{pred}(v_i)} \Gamma_u\Big) \tag{20}$$

where $\text{pred}(v_i)$ denotes the set of predecessor nodes of v_i in the DAG. Specifically, the reasoning function $\mathcal{G}(\cdot)$ is implemented through a forward pass of a language model, where the input consists of:

- **Sub-question** $\tilde{Q}_i$: The current sub-question to be answered at node v_i.
- **Predecessor outputs** $\bigoplus_{u \in \text{pred}(v_i)} S_u$: An aggregated representation of reasoning states from all preceding nodes of v_i, providing contextual information or intermediate results for reasoning.
- **Retrieved table regions** $\bigcup_{u \in \text{pred}(v_i)} \Gamma_u$: The union of table regions retrieved for each predecessor node, offering structured evidence to support the reasoning process.

The execution state S_i encapsulates the reasoning result at node v_i, which represents an intermediate result or a final answer. In Figure 2, node 3 performs reasoning based on node 1 and node 2, using their retrieved subtables to compute the number of wins for the *"SF 49ers"* as the home and away team.

The Reasoning Module treats each reasoning step as a self-contained operation that integrates prior outputs and retrieved evidence in a unified prompt. The Reasoning Module ensures that each node is executed only once, while still capturing dependency signals from the DAG structure. By embedding the necessary contextual information into the prompt, the LLM is guided to focus on localized reasoning, making the process efficient and well-aligned with the multi-stage plan.

After all reasoning nodes in the DAG are executed, the final answer is generated from the states of terminal nodes. We denote the set of such terminal nodes as V_{term}, and define the final answer A as the output of an answer generation function $\mathcal{A}(\cdot)$ applied over their reasoning states:

$$A = \mathcal{A}\Big(\{S_i \mid v_i \in V_{\text{term}}\}\Big) \tag{21}$$

The answer generation function synthesizes the final response by aggregating intermediate results from all relevant terminal nodes in the solution plan. As shown in Figure 2, the final answer *"Home games"* is derived from the state of terminal node 4.

The Reasoning Module enables structured and context-aware multi-step inference by executing each reasoning node based on its dependencies and retrieved evidence. The DAG-based design maintains logical consistency across steps while allowing each node to be executed independently in a single forward pass. By integrating intermediate outputs and localized table regions into unified prompts, the module delivers accurate reasoning, making it highly adaptable to large, complex table question-answering scenarios.

3.4 Overall Algorithm

The overall algorithm of AixelAsk, is illustrated in Algorithm 1.

For each sample, we take the question Q and table T as input. In Decomposition Module, we first extract the table metadata $M(T)$, construct a DAG-based solution plan $G = (V, E)$ and then refine each sub-question via schema linking based on Eq. 1–6. In addition to question decomposition, we also perform table decomposition into row and column chunks based on Eq. 10. In Retrieval Module, for each execution node labeled as *Retrieval*, we identify relevant table content using a row-column level ensemble strategy based on Eq. 11-18. The retrieved rows and columns are then mapped back to the original table to construct a compact sub-table based on Eq.19. In the Reasoning Module, for each execution node labeled as *Reasoning*, we infer its state based on Eq.20. After all nodes in the

Algorithm 1 Overall Algorithm

Input: The question Q and a large table T.
Output: The final answer A.
Extract metadata of table T based on Eq. 1.
Generate the solution plan and construct the DAG $G = (V, E)$ using Eq. 2.
for each node $v = (Q, \alpha)$ in G **do**
  Apply schema-aware refinement to Q to obtain $\tilde{Q}$ and update $v = (\tilde{Q}, \alpha)$ using Eq. 6.
end for
Decompose the table into row and column chunks to construct $C(T)$ based on Eq. 10.
for each node $v = (\tilde{Q}, \alpha)$ in G **do**
  if α = Retrieval **then**
    Perform semantic similarity matching for $\tilde{Q}$ based on Eq. 11-15.
    Perform keyword matching for $\tilde{Q}$ based on Eq. 16-17.
    Construct sub-table Γ by merging results and mapping them using Eq. 18- 19.
  else if α = Reasoning **then**
    Update the node state using the state of predecessor nodes and the retrieved table regions based on Eq. 20.
  end if
end for
Generate the answer A based on the states of all terminal nodes in G using Eq. 21.
Return A.

DAG-based solution plan G are processed, the final answer A is derived by aggregating the states of all terminal nodes as Eq.21.

4 Experiments

In this section, we present the experimental evaluation of AixelAsk. We first introduce the datasets, baselines, and evaluation metrics used in our experiments. We then present the experimental studies to evaluate the effectiveness and cost-efficiency of AixelAsk. Additionally, we conduct ablation studies to investigate the contributions of individual components. We further conduct scalability experiments to examine how AixelAsk performs as table size increases. Finally, we design a study comparing different RAG strategies to validate the advantages of our proposed row-column RAG.

4.1 Experimental Setup

4.1.1 Datasets. We conduct experiments on five datasets, with detailed statistics presented in Table 1.

- **WikiTQ-4k**: WikiTableQuestions (WikiTQ) [40] is a widely used benchmark dataset designed for question answering over structured tabular data. For our experiments, we filter the dataset to include only tables with more than 4k tokens, creating a subset of 488 entries.
- **WikiTQ+**: To augment the WikiTQ dataset, we extend the medium-sized tables (>3k tokens) by adding rows using the GPT-4o model with prompt-based table generation, expanding the table size to at least 8k tokens. The original answers are preserved and remain valid after table expansion.

- **TabFact+**: TabFact [11] dataset offers domain-diverse tables paired with fact-checking statements. However, the original TabFact tables are typically small. We use the same way to extend medium-sized tables by adding rows, increasing each table's size to over 8k tokens while preserving factual consistency.
- **WikiSQL+**: WikiSQL [64] is a dataset for natural language to SQL parsing. We execute the original SQL queries to obtain answers, retain tables with over 3k tokens and string-form outputs, and expand each table to over 8k tokens. Through this process, we obtain WikiSQL+, a dataset adapted for our task.
- **FeTaQA+**: FeTaQA [38] is a free-form table question answering dataset. However, the tables are relatively small. To augment the FeTaQA dataset, we expand the medium-sized tables to over 8k tokens by adding rows, while ensuring that the original answers remain valid.

Table 1. Detailed Statistics for Our Evaluation Datasets

Dataset	Size	Tbl Tk.	Rows	Cols	Q Tk.
WikiTQ-4k	488	7634.45	206.14	6.78	13.05
WikiTQ+	1023	8418.80	287.38	6.89	12.78
TabFact+	255	8849.75	135.48	9.81	23.00
WikiSQL+	235	10333.74	315.95	6.41	16.45
FeTaQA+	348	9053.64	156.11	10.50	18.67

Table 2. Performance Comparison on Proprietary LLMs across Four Datasets (Exact Match Score)

	Method	WikiTQ-4k (%)		WikiTQ+ (%)		TabFact+ (%)		WikiSQL+ (%)	
PLM	Tapas	2.94		2.12		46.53		4.35	
		GPT-3.5	**GPT-4o**	**GPT-3.5**	**GPT-4o**	**GPT-3.5**	**GPT-4o**	**GPT-3.5**	**GPT-4o**
Code-Driven	DIN-SQL + QA	15.88	20.59	10.42	17.46	60.40	68.32	76.81	79.71
	Binder	16.47	19.41	8.47	13.23	55.45	60.40	71.01	75.36
	ReAcTable	31.76	37.06	10.05	15.87	67.33	70.30	73.91	75.36
LLM	Few-shot QA	32.35	50.59	24.34	28.04	61.39	67.33	57.97	63.77
	CoT Prompting	30.59	47.06	33.33	39.68	68.32	72.28	69.57	71.01
	Dater	35.29	42.94	22.75	30.16	67.33	71.29	53.62	63.77
	Chain-of-Table	38.23	49.41	26.46	30.69	69.31	73.27	75.36	78.26
	TableRAG	40.59	50.59	36.51	41.27	70.30	72.28	79.71	81.15
	DocETL	25.88	27.65	20.11	24.87	57.43	60.40	55.07	60.87
	LOTUS	33.53	39.41	25.40	29.10	56.44	58.97	62.32	65.22
	Galois	31.18	41.18	24.87	28.57	61.39	67.33	66.67	74.31
	AixelAsk	**50.00**	**57.65**	**43.39**	**48.15**	**74.26**	**79.21**	**82.61**	**86.96**

4.1.2 Baselines and Metrics. We compare our AixelAsk with two categories of baselines: Executable Code-driven Methods and LM-based Methods. For the Executable Code-driven Methods, we select DIN-SQL, Binder, and ReAcTable. For LM-based Methods, we select Tapas as PLM-based approach, and select Few-shot QA, CoT Prompting, Dater, Chain-of-Table, TableRAG as LLM-based approaches. Additionally, we include DocETL, LOTUS and Galois as baseline systems powered by LLMs. Except for Few-shot QA and Chain-of-Thought, which we re-implemented, all other baselines are evaluated using their publicly available official implementations.

- **Tapas** [15] is a BERT-based Table QA model trained with weak supervision to predict answers by selecting table cells and applying aggregation without generating logical forms.
- **DIN-SQL** [43] translates natural language questions into SQL queries by decomposing the task into sub-problems, enabling step-by-step reasoning with LLMs. After generating the SQL query, we execute it and perform an additional QA step to produce the final answer.
- **Binder** [12] is a training-free neural-symbolic framework that combines language model capabilities with programming constructs to parse and execute programs for complex table-based questions.
- **ReAcTable** [59] is a table question-answering framework that leverages SQL and Python executors to iteratively transform data for complex reasoning without fine-tuning.
- **Few-shot QA** is a method that leverages the advanced capabilities of LLMs in a few-shot learning setting for Table QA.
- **CoT Prompting** [51] is a prompting strategy that guides the LLM to reason step by step when answering complex questions.
- **Dater** [54] is a table-based reasoning framework that uses LLMs to decompose tables and questions into sub-evidence and sub-questions through a "parsing-execution-filling" approach.
- **Chain-of-Table** [52] is a framework for table-based reasoning that guides LLMs to iteratively generate and update tables as a reasoning chain, enabling structured representation of intermediate steps for accurate predictions.
- **TableRAG** [8] is a retrieval-augmented framework for large-scale table understanding that combines query expansion with schema and cell-level retrieval.
- **DocETL** [46] is a declarative, agent-based optimization framework for complex document processing tasks. We adapt it to our setting by flattening tables into plain-text documents as input.
- **LOTUS** [41] is a semantic data processing framework supporting declarative natural language operators and embedding-based retrieval over structured and unstructured data.
- **Galois** [45] is a query engine that executes SQL over LLMs with logical and physical optimization strategies. We prepend Galois_F with DIN-SQL in our implementation to convert natural language questions into SQL queries for execution.

We use the following metrics to assess the quality and efficiency of these methods, measured per full question response.

- **Exact Match (EM)**: The percentage of predictions that exactly match the ground truth. A prediction is considered correct only if all answers for that question are correct.
- **Rouge**[24] measures n-gram overlap between generated and reference texts. We use it when evaluating free-form answers response in the FeTaQA+ dataset.
- **LLM Token Usage**: The average number of tokens consumed per full question response, including input and output tokens.[2]
- **Latency**: The average time (in seconds) required to complete one question response, indicating the runtime efficiency of the framework.

4.2 Implementation Details

4.2.1 Hyperparameter Settings and Details. In our experiments, we fine-tune the hyperparameters and prompts for our method and baseline models using the validation dataset. For Tapas, we set the maximum sequence length to 4096 tokens and train for 30 epochs using a batch size of 2. The

[2]Input and output tokens are priced differently in most LLM APIs, so they are calculated separately. We use the cl100k_base tokenizer for token calculation.

AdamW optimizer [20] is used with a learning rate of 5e-5 and weight decay of 0.01. For LLM-based methods, the temperature is set to 0.1. In AixelAsk, the parameter t_o is set to 0.2.

In implementing AixelAsk, we fine-tune RoBERTa [29] on 500 manually annotated examples for reasoning type classification. For DAG-based decomposition, we use 15 few-shot examples (5 per reasoning type). All annotated examples are sourced from the training sets of the WikiTQ-4k, WikiTQ+, and TabFact+ datasets, ensuring no overlap with the test sets. During evaluation, the initial success rate of DAG generation is 93.7%. As shown in Figure 4, the DAG validity rate reaches 100% when max regeneration attempts is 4. Therefore, we set max regeneration attempts to 4. The runtime and token cost associated with DAG regeneration are included in our experimental results. Token cost for embedding generation is a one-time overhead and is not included in the experimental results.

4.2.2 Experimental Environment. The experiments are conducted on a server equipped with an Intel(R) Xeon(R) W-2275 CPU @3.30GHz (14 cores), 512GB memory, and 2 NVIDIA RTX A6000 GPUs. The fine-tuning of Tapas is implemented in PyTorch 1.13.0 with CUDA 11.6 support. We adopt GPT-3.5 and GPT-4o as proprietary LLMs, and QWen-7B, Mistral-7B as open-source models deployed locally, for both baseline methods and AixelAsk. Additionally, AixelAsk utilizes the *text-embedding-large-3* as embedding model, which shows superior performance in our preliminary experiments.

4.3 Comparison with Baselines

In this section, we present a comparative analysis between AixelAsk and various baselines across five datasets, demonstrating the effectiveness and efficiency of AixelAsk in handling the Large Table QA task.

4.3.1 Effectiveness Evaluation. We compare the performance of AixelAsk with baseline methods to demonstrate its effectiveness in the Large Table QA task. Tables 2 and 3 present the results of AixelAsk and all baselines on four datasets using proprietary and open-source LLMs, respectively. Table 4 shows the results of AixelAsk and all baselines on FeTaQA+ dataset.

Table 3. Performance Comparison on Open-Source LLMs across Four Datasets (Exact Match Score)

	Method	**WikiTQ-4k (%)**		**WikiTQ+ (%)**		**TabFact+ (%)**		**WikiSQL+ (%)**	
PLM	Tapas	2.94		2.12		46.53		4.35	
		QWen	**Mistral**	**QWen**	**Mistral**	**QWen**	**Mistral**	**QWen**	**Mistral**
Code-Driven	DIN-SQL + QA	22.64	24.53	13.76	19.05	61.39	63.37	68.12	72.46
	Binder	29.41	37.06	8.99	16.40	59.41	56.44	63.79	68.12
	ReAcTable	27.06	25.29	17.26	18.52	66.34	64.36	69.57	71.01
LLM	Few-shot QA	26.42	29.25	11.11	14.81	60.40	55.45	50.72	55.07
	CoT Prompting	28.24	31.18	25.93	22.22	67.33	57.43	63.77	66.67
	Dater	31.13	30.19	17.99	19.05	64.36	65.35	50.73	59.42
	Chain-of-Table	26.47	28.23	17.46	21.69	58.42	62.38	75.36	73.91
	TableRAG	30.59	33.53	23.28	25.93	70.30	69.31	76.81	73.91
	DocETL	22.94	24.12	16.93	19.58	54.46	58.42	53.62	56.52
	LOTUS	29.41	27.06	19.05	20.11	60.40	61.39	55.97	57.97
	Galois	25.88	28.82	20.63	21.16	63.37	59.41	65.21	68.12
	AixelAsk	**35.88**	**40.59**	**26.98**	**32.80**	**71.29**	**73.27**	**78.26**	**81.16**

Table 4. Performance Comparison on FeTaQA+ Dataset (Rouge-1|2|L Score)

	Method	GPT-3.5	GPT-4o	QWen	Mistral
PLM	Tapas	4.11 | 0.97 | 2.73			
Code-Driven	DIN-SQL + QA	10.96 | 4.98 | 6.55	15.49 | 6.12 | 8.52	9.38 | 2.37 | 4.61	12.72 | 8.59 | 11.76
	Binder	11.73 | 3.82 | 9.24	15.56 | 5.45 | 11.98	8.51 | 2.94 | 5.98	17.51 | 14.83 | 12.43
	ReAcTable	26.49 | 12.82 | 16.97	29.60 | 17.64 | 24.04	23.88 | 13.35 | 17.34	22.39 | 12.46 | 19.79
LLM	Few-shot QA	21.40 | 10.88 | 13.11	26.18 | 12.84 | 23.81	18.93 | 7.71 | 12.71	14.92 | 8.83 | 11.47
	CoT Prompting	41.92 | 21.05 | 35.34	45.79 | 25.62 | 39.69	39.47 | 18.62 | 32.95	36.84 | 16.74 | 30.12
	Dater	28.76 | 13.87 | 19.73	30.33 | 24.95 | 27.23	26.18 | 10.38 | 16.97	23.55 | 15.13 | 19.84
	Chain-of-Table	40.16 | 15.41 | 30.64	46.26 | 21.71 | 29.24	38.48 | 11.84 | 28.22	40.76 | 17.42 | 29.20
	TableRAG	50.55 | 28.53 | 35.73	53.42 | 31.64 | 37.59	48.92 | 24.02 | 31.79	45.25 | 17.31 | 31.89
	DocETL	16.02 | 7.46 | 11.34	23.64 | 5.60 | 13.11	17.15 | 6.64 | 12.71	14.14 | 2.58 | 10.51
	LOTUS	23.02 | 10.88 | 18.79	27.88 | 15.63 | 20.17	18.67 | 9.73 | 10.18	21.90 | 11.57 | 14.21
	Galois	33.58 | 15.93 | 21.77	38.30 | 17.34 | 24.39	31.95 | 14.02 | 20.07	30.75 | 12.15 | 24.23
	AixelAsk	**56.11 | 30.91 | 44.03**	**61.16 | 44.26 | 50.91**	**57.77 | 28.19 | 35.37**	**54.35 | 20.07 | 34.67**

Note: The Rouge metrics are reported in the order of Rouge-1, Rouge-2, and Rouge-L.

Table 2 shows that AixelAsk outperforms PLM-based, executable code-driven, and LLM-based methods on four datasets when using proprietary LLMs (GPT-3.5 and GPT-4o), demonstrating its effectiveness in handling large tables. Focusing on the open-source LLM results, as shown in Table 3, AixelAsk consistently outperforms all baselines across QWen-7B and Mistral-7B on the same four datasets, confirming its generalizability across different LLM backbones. In the free-form answer setting on FeTaQA+, as shown in Table 4, AixelAsk achieves the highest Rouge scores (Rouge-1, Rouge-2, and Rouge-L), further validating its effectiveness in large table QA tasks.

Compared to the best-performing baseline (TableRAG), AixelAsk achieves notable improvements, with gains of up to 7.06% on WikiTQ-4k, 6.88% on WikiTQ+, and 5.81% on WikiSQL+ when using GPT-4o. On TabFact+, AixelAsk also achieves the highest EM score of 78.84% with GPT-4o, outperforming Chain-of-Table by 5.94%, demonstrating strong reasoning capabilities even in large-table fact verification tasks. On the FeTaQA+ dataset, which requires free-form answers, AixelAsk outperforms all baselines, achieving gains of 7.74% over TableRAG and 14.9% over Chain-of-Table in Rouge-1 score with GPT-4o.

Overall, our proposed AixelAsk demonstrates consistent effectiveness across all five datasets and backbone LLMs evaluated. We also observe that open-source models yield relatively lower performance compared to proprietary ones, indicating that AixelAsk, like most existing methods, still relies on the underlying reasoning capability of the LLM. However, AixelAsk shows greater advantage in enhancing the reasoning performance of open-source LLMs through structured decomposition and targeted retrieval. Consequently, AixelAsk offers an effective framework for the Large Table QA task.

4.3.2 Efficiency Evaluation. We evaluate the efficiency of AixelAsk compared to baseline methods on the five datasets using GPT-4o. The results are shown in Table 5, focusing on two primary metrics, Token Usage and Latency.

Cost-Efficiency. In terms of token usage, AixelAsk consistently achieves the lowest input token count and relatively low output token usage across all five datasets, utilizing fewer than 5.2k input tokens and around 300 output tokens on average. The significant reduction in input tokens marks a clear advantage over baselines like ReAcTable (over 20k) and Dater (exceeding 330k). Even compared to the Few-shot QA and COT prompting method, which maintains extremely low output token usage but relatively high input tokens, AixelAsk provides a more balanced and

overall reduced token footprint. The response size of the Retrieval Module typically consists of 8–40 cells, totaling approximately 50–200 tokens. This small response size further underscores the cost-effectiveness of our approach. These results underline the effectiveness of AixelAsk in significantly reducing input token consumption, demonstrating its superior cost-efficiency without sacrificing accuracy.

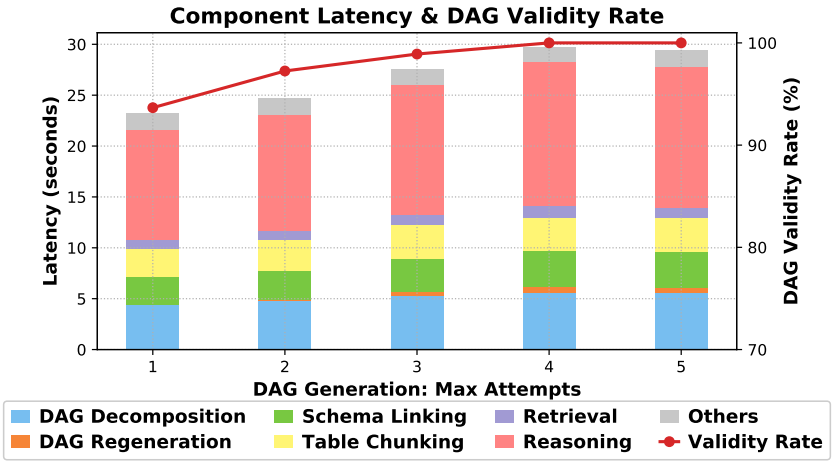


Fig. 4. AixelAsk's Latency Breakdown and DAG Validity Rate across Different max Generation Attempts

Time-Efficiency. Regarding latency, AixelAsk demonstrates strong efficiency, averaging only 29.4 seconds per instance, which is the third-lowest among all baselines. Compared to methods like Dater (678.2 seconds) and Chain-of-Table (43.4 seconds), which suffer from higher latency due to multiple LLM invocations during iterative reasoning steps, AixelAsk substantially reduces computational overhead by requiring only 4.69 LLM calls per question on average. While Few-shot QA and CoT prompting are faster due to using only a single LLM invocation, their accuracy is significantly lower. Figure 4 shows the latency breakdown of each component in AixelAsk under different maximum DAG generation attempts. The majority of time is spent on Reasoning, as it involves multiple LLM calls. The relatively low time spent on handling invalid DAG demonstrates the robustness of AixelAsk. Therefore, considering both accuracy and inference speed, AixelAsk achieves an advantageous combination of performance and efficiency, making it highly suitable for practical deployment in Large Table QA scenarios.

In conclusion, AixelAsk demonstrates both token-efficiency and time-efficiency while maintaining high reasoning accuracy, making it well-suited for the Large Table QA task.

4.4 Different RAG Strategy Study

To evaluate the effectiveness of the proposed Row-Column RAG strategy, we compare it with several alternative RAG approaches using GPT-4o on the WikiTQ-4k, WikiTQ+, and TabFact+ datasets. Specifically, we integrate different RAG strategies with the few-shot QA and compare their QA accuracy and input token cost. The following RAG strategies are considered:

- **Text-RAG** retrieves the table as plain text, ignoring structure.
- **Column-RAG** retrieves columns via schema-level matching.
- **Row-RAG** retrieves rows using semantic similarity with the question.
- **Cell-RAG** retrieves cells based on semantic similarity.

Table 5. Comparison of Average Token Usage and Latency Across Different Methods on Five Datasets

	Method	WikiTQ-4k		WikiTQ+		TabFact+		WikiSQL+		FeTaQA+		Latency(s)
		I	O	I	O	I	O	I	O	I	O	
Code-Driven	DIN-SQL	9792	1382	9744	1582	9277	1321	10783	1027	9974	1575	196.2
	Binder	9402	784	9320	881	7320	963	8937	801	10233	1144	110.7
	ReAcTable	42021	232	35232	742	30428	596	19284	630	47922	890	89.3
LLM	Few-shot QA	8962	**42**	9457	**53**	10300	**1**	10118	**28**	8840	**94**	**7.2**
	CoT Prompting	9685	212	10180	289	10199	163	10840	216	9561	382	10.7
	Dater	330294	4680	360584	4916	388581	2974	372469	2663	416629	4772	649.8
	Chain-of-Table	20985	189	22615	275	29360	495	13612	401	23511	1082	56.6
	TableRAG	9652	632	13271	733	14007	873	10331	322	13364	1577	36.3
	DocETL	13846	783	16243	717	16388	979	17202	560	15712	2220	50.8
	LOTUS	9586	664	11834	855	11196	901	14354	519	18379	1427	61.9
	Galois	11821	1644	14459	1867	13150	1076	18025	1472	14613	1913	33.8
	AixelAsk	**3176**	188	**4427**	219	**4798**	247	**5171**	266	**4934**	681	29.4

Token Usage: Avg. Input(I)/Output(O) tokens per entry; **Latency**: Avg. processing time per entry.

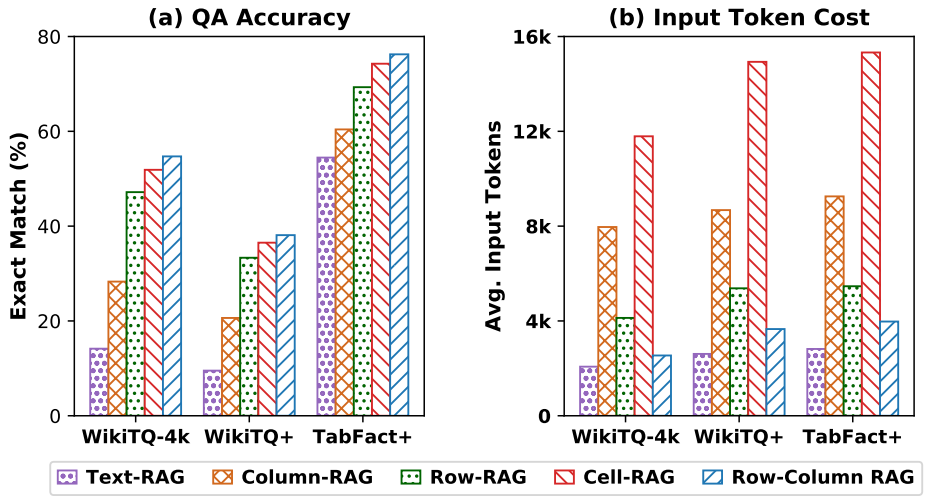


Fig. 5. Comparison of QA Accuracy and Token Cost across Different RAG Strategies

Figure 5 illustrates the QA accuracy and input token cost of different RAG strategies. In Figure 5(a), our proposed **Row-Column RAG** achieves the highest QA performance across three datasets, highlighting its effectiveness for Large Table QA tasks. In Figure 5(b), our RAG strategy also achieves the second-lowest token usage, significantly reducing costs while maintaining strong performance. While Text-RAG uses the fewest tokens, its lack of structural awareness often results in poor retrieval quality. In contrast, our Row-Column RAG leverages both row and column contexts, preserving the table's structure and forming coherent subtables. This structured retrieval guides the LLM focus directly on critical information, improving both retrieval quality and cost-efficiency.

4.5 Ablation Study

To evaluate the effectiveness and efficiency of individual modules in AixelAsk, we conduct ablation study on WikiTQ-4k, WikiTQ+ and TabFact+ datasets. We design the following variants of AixelAsk:

- **AixelAsk w/o Decomposition Module**: In this variant, the Decomposition Module is removed, and AixelAsk directly performs a single round of retrieval and reasoning on the original question. It skips the DAG-based decomposition of complex questions with schema linking, resulting in a lack of intermediate guidance during the reasoning process of the LLMs.
- **AixelAsk w/o Retrieval Module**: In this variant, AixelAsk does not perform row-column ensemble retrieval to construct sub-tables for answering question. Instead, the large table is directly input into the model for processing each sub-question.

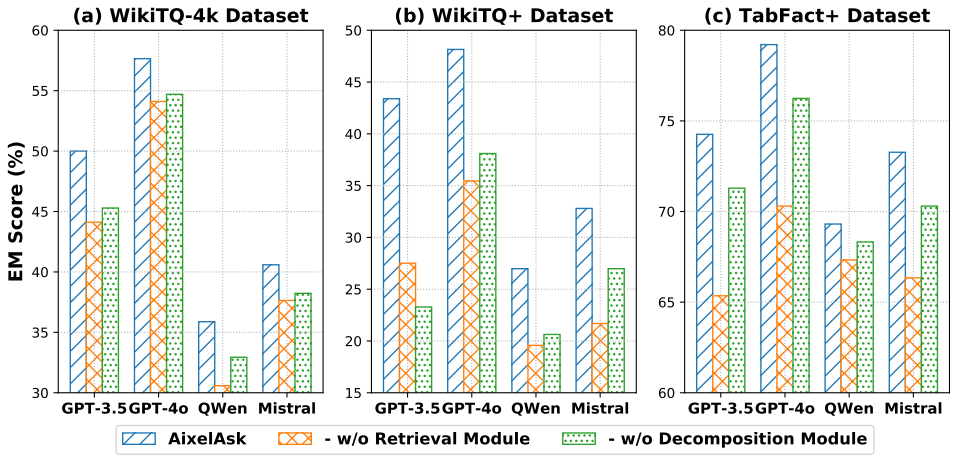


Fig. 6. Effectiveness Evaluation Result in Ablation Study

4.5.1 Module Effectiveness Evaluation. We evaluate the contribution of each module in AixelAsk to overall performance. As shown in Figure 6, AixelAsk outperforms all ablation variants across different LLMs and datasets, highlighting the importance of its modular design. Among all components, the Retrieval Module contributes most to the performance by addressing the overwhelming size of input data, a key challenge in Large Table QA. The Retrieval Module extracts the most relevant rows and columns, reducing the large table into smaller, focused sub-tables and ensuring that the LLM processes only the most critical information, which leads to significant improvements in accuracy.

Additionally, the Decomposition Module has a notable impact on performance. By constructing a DAG-based solution plan, it organizes intricate questions into logically structured sub-questions, reducing the reasoning burden on LLMs. Schema linking further enhances its effectiveness by aligning the decomposition with table structure and data relationships.

While the Retrieval Module ensures effectiveness by narrowing down the table scope to critical information, the Decomposition Module facilitates effective reasoning over complex queries by breaking them down into sub-questions. The combination of these two modules enables AixelAsk to achieve the best performance, demonstrating the effectiveness of its modular design and its capability to handle the Large Table QA task with high accuracy.

4.5.2 Module Efficiency Analysis. We evaluate the cost-efficiency of AixelAsk and its ablation variants by analyzing token usage on three datasets using GPT-4o. Since each module naturally introduces processing latency, we focus solely on token-related costs.

As shown in Figure 7, the cost of AixelAsk w/o Retrieval Module is the highest, reaching average 10.23 times that of the full AixelAsk across three datasets. This highlights the critical role of the

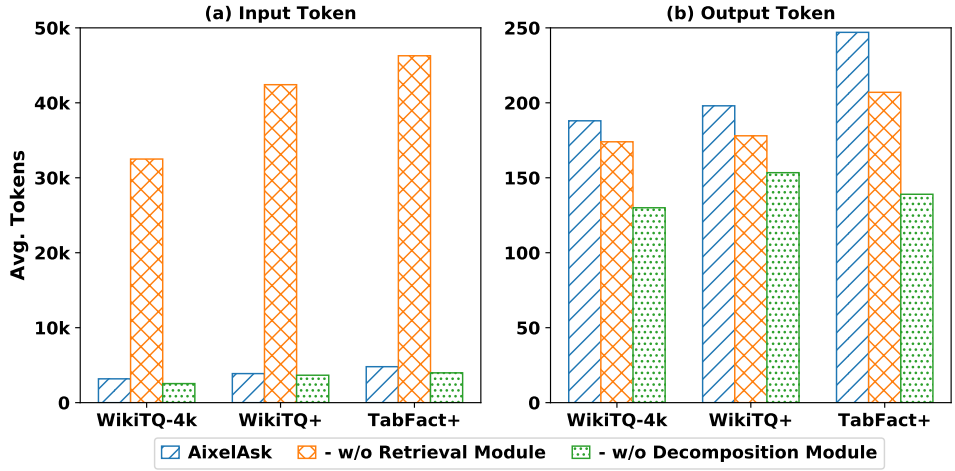


Fig. 7. Cost-Efficiency Evaluation Result in Ablation Study

Retrieval Module in reducing costs. By efficiently extracting and remapping the most relevant rows and columns into smaller sub-tables, the Retrieval Module significantly minimizes the size of input data passed to the LLM. This reduction not only improves processing efficiency but also limits the number of tokens processed by the model, thereby lowering the overall computational cost.

The cost of AixelAsk is the second lowest, slightly higher than AixelAsk w/o Decomposition Module due to the additional LLM calls for generating the solution plan. However, this added cost is justified by the Decomposition Module's significant accuracy improvement, as it enhances reasoning performance by decomposing question into manageable sub-questions, making the additional cost worthwhile. Overall, AixelAsk shows strong cost-efficiency in the ablation study, balancing high accuracy and low cost.

4.6 Scalability Experiment

To evaluate the scalability of AixelAsk in handling large tables, we conduct experiments on a combined version of the WikiTQ and WikiTQ+ datasets, the TabFact and TabFact+ datasets, as well as the WikiSQL dataset. For WikiSQL, we construct varying-size subsets by adding rows to tables of different sizes. For clear and meaningful comparisons, we exclude scalability results for baselines such as DIN-SQL, Few-shot QA, and DocETL, as their overall accuracy is relatively low. Specifically, each dataset is partitioned into subsets based on table token size, with 2K-token increments, to assess performance as table size increases.

The comparison reveals how performance varies with increasing table size. As shown in Figure 8, most baseline methods experience a sharp drop in accuracy when table size exceeds 2k or 4k tokens. The sharp decline observed in baselines is primarily due to the limited context length that LLMs can handle, which leads to truncation or omission of critical information in large tables. In contrast, AixelAsk is the most stable among all baselines, maintaining relatively high accuracy with the smallest performance drop as table size increases, demonstrating superior scalability in large table scenarios. This improved scalability is primarily attributed to our retrieval strategy, which identifies relevant sub-table, reducing the reasoning burden on the LLM as table size increases. Additionally, the decomposition strategy segments complex questions into structured sub-questions, further enhancing scalability.

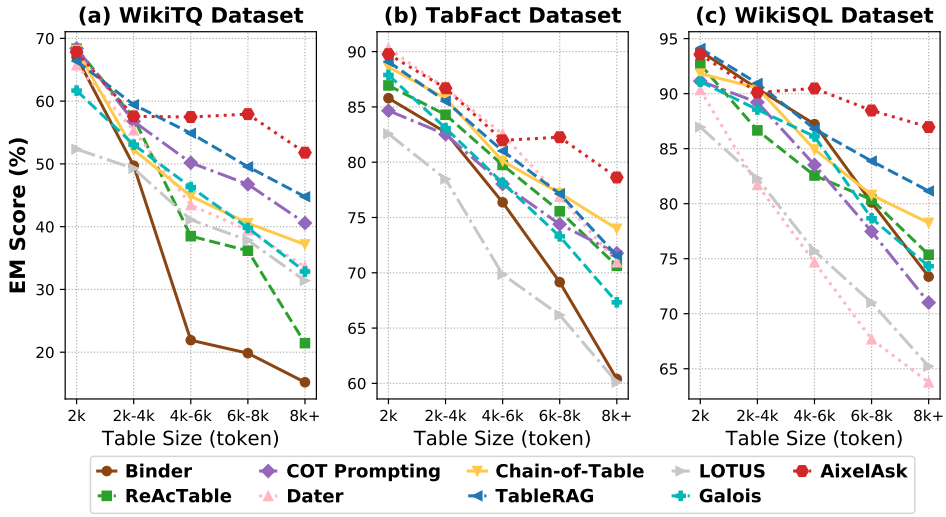


Fig. 8. Scalability of Different Methods on Large Table with Varying Table Size

5 Conclusions

In this paper, we propose AixelAsk, a novel framework for Large Table QA that achieves high accuracy while significantly reducing token cost and inference overhead. We first design a DAG-based decomposition method to break down questions into structured sub-questions with explicit execution dependencies, enabling clear and stepwise reasoning. We then introduce a row-column level ensemble retrieval strategy to extract the most relevant table regions, reduce input tokens, and guide the LLM to focus on critical content. Finally, AixelAsk synthesizes the retrieved information and intermediate reasoning results to derive the final answer. Extensive experiments on three large-table QA datasets demonstrate the superiority of AixelAsk over state-of-the-art baselines in both accuracy and efficiency.

Acknowledgments

This work is supported by National Natural Science Foundation of China (U2441237). Zhaojing Luo was supported by National Key Research and Development Program of China (2024YFC3308200), National Natural Science Foundation of China (62472030).

References

[1] Haoqiong Bian, Ying Yan, Wenbo Tao, Liang Jeff Chen, Yueguo Chen, Xiaoyong Du, and Thomas Moscibroda. 2017. Wide Table Layout Optimization based on Column Ordering and Duplication. In *Proceedings of the ACM International Conference on Management of Data*. ACM, 299–314.

[2] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2SQL is Not Enough: Unifying AI and Databases with TAG. *arXiv preprint arXiv:2408.14717* (2024).

[3] Jan-Micha Bodensohn and Carsten Binnig. 2024. Rethinking Table Retrieval from Data Lakes. In *Proceedings of the International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. ACM, 2:1–2:5.

[4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2206–2240.

[5] Douglas Burdick, Marina Danilevsky, Alexandre V Evfimievski, Yannis Katsis, and Nancy Wang. 2020. Table extraction and understanding for scientific and enterprise applications. *Proceedings of the VLDB Endowment* 13, 12 (2020),

3433–3436.

[6] Yihan Cao, Shuyi Chen, Ryan Liu, Zhiruo Wang, and Daniel Fried. 2023. API-Assisted Code Generation for Question Answering on Varied Table Structures. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 14536–14548.

[7] Saneem A. Chemmengath, Vishwajeet Kumar, Samarth Bharadwaj, Jaydeep Sen, Mustafa Canim, Soumen Chakrabarti, Alfio Gliozzo, and Karthik Sankaranarayanan. 2021. Topic Transferable Table Question Answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 4159–4172.

[8] Si-An Chen, Lesly Miculicich, Julian Martin Eisenschlos, Zifeng Wang, Zilong Wang, Yanfei Chen, Yasuhisa Fujii, Hsuan-Tien Lin, Chen-Yu Lee, and Tomas Pfister. 2024. TableRAG: Million-Token Table Understanding with Language Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 74899–74921.

[9] Wenhu Chen. 2023. Large Language Models are few-shot Table Reasoners. In *Findings of the European Chapter of the Association for Computational Linguistics*. 1120–1130.

[10] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2020. TabFact: A Large-scale Dataset for Table-based Fact Verification. In *Proceedings of the International Conference on Learning Representations*.

[11] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2020. Tabfact: A large-scale dataset for table-based fact verification. In *Proceedings of the International Conference on Learning Representations*.

[12] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding Language Models in Symbolic Languages. *Proceedings of the International Conference on Learning Representations* abs/2210.02875 (2023).

[13] Zhangtao Cheng, Jienan Zhang, Xovee Xu, Goce Trajcevski, Ting Zhong, and Fan Zhou. 2024. Retrieval-augmented hypergraph for multimodal social media popularity prediction. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 445–455.

[14] Zihui Gu, Ju Fan, Nan Tang, Lei Cao, Bowen Jia, Sam Madden, and Xiaoyong Du. 2023. Few-shot text-to-sql translation using structure and content prompt learning. *Proceedings of the ACM International Conference on Management of Data* 1, 2 (2023), 1–28.

[15] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pre-training. In *Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 4320–4333.

[16] Zezhou Huang, Pavan Kalyan Damalapati, and Eugene Wu. 2023. Data Ambiguity Strikes Back: How Documentation Improves GPT's Text-to-SQL. In *NeurIPS 2023 Second Table Representation Learning Workshop*.

[17] Shayekh Bin Islam, Md Asib Rahman, K S M Tozammel Hossain, Enamul Hoque, Shafiq Joty, and Md Rizwan Parvez. 2024. Open-RAG: Enhanced Retrieval Augmented Reasoning with Open-Source Large Language Models. In *Findings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 14231–14244.

[18] Gautier Izacard and Edouard Grave. 2021. Distilling Knowledge from Reader to Retriever for Question Answering. In *Proceedings of the International Conference on Learning Representations*.

[19] Pankaj Joshi, Aditya Gupta, Pankaj Kumar, and Manas Sisodia. 2024. Robust multi model rag pipeline for documents containing text, table & images. In *Proceedings of the International Conference on Applied Artificial Intelligence and Computing*. IEEE, 993–999.

[20] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[21] Flip Korn, Xuezhi Wang, You Wu, and Cong Yu. 2019. Automatically generating interesting facts from wikipedia tables. In *Proceedings of the ACM International Conference on Management of Data*. 349–361.

[22] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM International Conference on Management of Data* 2, 3 (2024), 1–28.

[23] Yiming Li, Yanyan Shen, and Lei Chen. 2022. Camel: Managing data for efficient stream learning. In *Proceedings of the ACM International Conference on Management of Data*. 1271–1285.

[24] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.

[25] Weizhe Lin, Rexhina Blloshmi, Bill Byrne, Adrià de Gispert, and Gonzalo Iglesias. 2023. LI-RAGE: Late Interaction Retrieval Augmented Generation with Explicit Signals for Open-Domain Table Question Answering. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 1557–1566.

[26] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)* (2025).

[27] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[28] Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. 2022. TAPEX: Table Pre-training via Learning a Neural SQL Executor. In *Proceedings of the International Conference on Learning Representations*.

[29] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR* abs/1907.11692 (2019). arXiv:1907.11692

[30] Zhaojing Luo, Shaofeng Cai, Gang Chen, Jinyang Gao, Wang-Chien Lee, Kee Yuan Ngiam, and Meihui Zhang. 2021. Improving Data Analytics with Fast and Adaptive Regularization. *IEEE Transactions on Knowledge and Data Engineering* 33, 2 (2021), 551–568.

[31] Zhaojing Luo, Shaofeng Cai, Can Cui, Beng Chin Ooi, and Yang Yang. 2021. Adaptive Knowledge Driven Regularization for Deep Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 8810–8818.

[32] Zhaojing Luo, Shaofeng Cai, Jinyang Gao, Meihui Zhang, Kee Yuan Ngiam, Gang Chen, and Wang-Chien Lee. 2018. Adaptive Lightweight Regularization Tool for Complex Analytics. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. 485–496.

[33] Zhaojing Luo, Shaofeng Cai, Yatong Wang, and Beng Chin Ooi. 2023. Regularized pairwise relationship based analytics for structured data. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[34] Zhaojing Luo, Sai Ho Yeung, Meihui Zhang, Kaiping Zheng, Lei Zhu, Gang Chen, Feiyi Fan, Qian Lin, Kee Yuan Ngiam, and Beng Chin Ooi. 2021. MLCask: Efficient management of component evolution in collaborative data analytics pipelines. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1655–1666.

[35] Ben Mann, N Ryder, M Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1 (2020).

[36] Nicholas Matsumoto, Jay Moran, Hyunjun Choi, Miguel E Hernandez, Mythreye Venkatesan, Paul Wang, and Jason H Moore. 2024. KRAGEN: a knowledge graph-enhanced RAG framework for biomedical problem solving using large language models. *Bioinformatics* 40, 6 (2024).

[37] Md Mahadi Hasan Nahid and Davood Rafiei. 2024. TabSQLify: Enhancing Reasoning Capabilities of LLMs Through Table Decomposition. In *Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 5725–5737.

[38] Linyong Nan, Chiachun Hsieh, Ziming Mao, Xi Victoria Lin, Neha Verma, Rui Zhang, Wojciech Kryściński, Hailey Schoelkopf, Riley Kong, Xiangru Tang, Mutethia Mutuma, Ben Rosand, Isabel Trindade, Renusree Bandaru, Jacob Cunningham, Caiming Xiong, and Dragomir Radev. 2022. FeTaQA: Free-form Table Question Answering. *Transactions of the Association for Computational Linguistics* 10 (2022), 35–49.

[39] Vaishali Pal, Andrew Yates, Evangelos Kanoulas, and Maarten de Rijke. 2023. MultiTabQA: Generating Tabular Answers for Multi-Table Question Answering. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 6322–6334.

[40] Panupong Pasupat and Percy Liang. 2015. Compositional Semantic Parsing on Semi-Structured Tables. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 1470–1480.

[41] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. *arXiv preprint arXiv:2407.11418* (2024).

[42] Nisarg Patel, Mohith Kulkarni, Mihir Parmar, Aashna Budhiraja, Mutsumi Nakamura, Neeraj Varshney, and Chitta Baral. 2024. Multi-LogiEval: Towards Evaluating Multi-Step Logical Reasoning Ability of Large Language Models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 20856–20879.

[43] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems* 36 (2023), 36339–36348.

[44] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the text-to-sql capabilities of large language models. *arXiv preprint arXiv:2204.00498* (2022).

[45] Dario Satriani, Enzo Veltri, Donatello Santoro, Sara Rosato, Simone Varriale, and Paolo Papotti. 2025. Logical and Physical Optimizations for SQL Query Execution over Large Language Models. *Proceedings of the ACM International Conference on Management of Data* 3, 3 (2025), 1–28.

[46] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2024. Docetl: Agentic query rewriting and evaluation for complex document processing. *arXiv preprint arXiv:2410.12189* (2024).

[47] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Richard James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. 2024. REPLUG: Retrieval-Augmented Black-Box Language Models. In *Proceedings of the North American Chapter of the Association for Computational Linguistics*. 8364–8377.
[48] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2024. Table meets llm: Can large language models understand structured table data? a benchmark and empirical study. In *Proceedings of the ACM International Conference on Web Search and Data Mining*. 645–654.
[49] Arif Usta and Semih Salihoglu. 2023. To Join or Not to Join: An Analysis on the Usefulness of Joining Tables in Open Government Data Portals. In *International Conference on Very Large Data Bases*.
[50] Enzo Veltri, Gilbert Badaro, Mohammed Saeed, and Paolo Papotti. 2023. Data Ambiguity Profiling for the Generation of Training Examples. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 450–463.
[51] Xuezhi Wang and Denny Zhou. 2024. Chain-of-Thought Reasoning Without Prompting. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 66383–66409.
[52] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2024. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. *Proceedings of the International Conference on Learning Representations* (2024).
[53] Kai Yang, Zhaojing Luo, Jinyang Gao, Junfeng Zhao, Beng Chin Ooi, and Bing Xie. 2021. Lda-reg: Knowledge driven regularization using external corpora. *IEEE Transactions on Knowledge and Data Engineering* 34, 12 (2021), 5840–5853.
[54] Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. 2023. Large Language Models are Versatile Decomposers: Decomposing Evidence and Questions for Table-based Reasoning. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*. Association for Computing Machinery, 174–184.
[55] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. 2020. TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 8413–8426.
[56] Meihui Zhang, Zhaoxuan Ji, Zhaojing Luo, Yuncheng Wu, and Chengliang Chai. 2024. Applications and Challenges for Large Language Models: From Data Management Perspective. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 5530–5541.
[57] Meihui Zhang, Liming Wang, Chi Zhang, and Zhaojing Luo. 2025. Aixel: A Unified, Adaptive and Extensible System for AI-powered Data Analysis. *arXiv preprint arXiv:2510.12642* (2025).
[58] Siyue Zhang, Anh Tuan Luu, and Chen Zhao. 2024. SynTQA: Synergistic Table-based Question Answering via Mixture of Text-to-SQL and E2E TQA. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2352–2364.
[59] Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh Patel. 2024. ReAcTable: Enhancing ReAct for Table Question Answering. *Proceedings of the VLDB Endowment* 17 (05 2024), 1981–1994.
[60] Bowen Zhao, Changkai Ji, Yuejie Zhang, Wen He, Yingwen Wang, Qing Wang, Rui Feng, and Xiaobo Zhang. 2023. Large Language Models are Complex Table Parsers. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 14786–14802.
[61] Yilun Zhao, Lyuhao Chen, Arman Cohan, and Chen Zhao. 2024. TaPERA: Enhancing faithfulness and interpretability in long-form table QA by content planning and execution-based reasoning. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 12824–12840.
[62] Yilun Zhao, Chen Zhao, Linyong Nan, Zhenting Qi, Wenlin Zhang, Xiangru Tang, Boyu Mi, and Dragomir Radev. 2023. RobuT: A Systematic Study of Table QA Robustness Against Human-Annotated Adversarial Perturbations. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 6064–6081.
[63] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103* (2017).
[64] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *CoRR* abs/1709.00103 (2017).

Received April 2025; revised July 2025; accepted August 2025